

전충명

Backend Developer · Cloud · AI Integration

010-8362-0913 zsc0248@naver.com github.com/Kade-Jeon wanted.co.kr/profile

2Y

Years Experience

3

Delivered Projects

2

Certifications

15+

Tech Stack

WORK EXPERIENCE

경력 사항

크리니티

2024.05 — 2026.01 · 1년 9개월

Backend Developer · Cloud

담당 업무

- Spring Boot 3.x / Java 기반 차세대 메일 서비스 백엔드 개발 참여
- ELK Stack 기반 중앙 집중형 모니터링 시스템 구축 및 운영 (KISTI 프라이빗 클라우드)
- JavaScript / JSP 풀스택 개발 및 고객사 맞춤 기능 구현, CI/CD 자동화 배포 관리
- KT Cloud / NCP 멀티 클라우드 환경에서 서비스 운영 및 배포 관리
- Spring AI 기반 공공 메일 서비스 AI 기능 PoC 설계 · 구현

주요 프로젝트 / 성과

01 Spring AI 기반 공공 메일 서비스 AI 기능 PoC

상황

레거시 메일 서비스의 AI 호출이 RestTemplate 기반이었고, 프롬프트가 하나의 텍스트로 합쳐져 전송되어 응답 품질이 낮았습니다. RAG · 에이전트 구조로의 확장도 불가능한 상태였습니다.

행동

회사 내 Java · Spring 기술 스택에 대한 선호가 강한 점을 감안해, Python 계열 프레임워크(LangChain 등) 대신 Spring AI 도입을 직접 제안했습니다. ChatClient 구조로 프롬프트 역할을 분리하고 1차로 번역 · 메일 작성 기능을 구현했으며, 이후 RAG · 에이전트 · Langfuse로 확장 가능한 아키텍처를 설계했습니다. 도입 제안부터 설계 · 구현까지 대부분을 담당했습니다.

결과

NIA(한국지능정보사회진흥원) 베타 운영에 도입되었습니다 (2026.01~). 단일 텍스트로 합쳐 전송하던 구조적 한계로 인해 프롬프트 엔지니어링을 반복해도 응답 품질이 오르지 않던 문제를 해소했고, ChatClient 역할 분리 구조 전환 후 응답 일관성이 눈에 띄게 향상되었습니다. G-cloud 공공 클라우드 메일의 최초 AI 기능으로 채택되었고, 이후 RAG 기능 추가 확장 요청이 들어왔지만 사전 설계 덕분에 재설계 없이 대응 가능한 상태입니다.

배운 점

아무리 프롬프트 엔지니어링을 반복해도 응답 품질이 오르지 않는 상황에서, 문제의 원인이 프롬프트 내용이 아니라 구조 자체에 있었음을 파악했습니다. 기능을 동작하게 만드는 것과 확장 가능하게 설계하는 것은 다른 문제임을 실감했고, 사전에 확장을 고려한 설계 덕분에 이후 RAG 기능 추가 요청에 재설계 없이 대응할 수 있었습니다. 기능 도입 시 확장 가능성까지 함께 설계하는 것이 실질적인 유지보수 비용을 줄인다는 것을 이 프로젝트에서 직접 확인했습니다.

02 KISTI 프라이빗 클라우드 ELK 모니터링 시스템 구축

상황

기존에는 Scouter로 일부 메트릭만 확인할 수 있었고, 로그를 보려면 관련 인원이 모든 VM에 root로 직접 접속해야 했습니다. 로그가 서버별로 분산되어 장애 원인 파악이 느렸고, 보안·운영 리스크도 있었습니다.

행동

Scouter 대비 로그·트랜잭션·슬로우쿼리까지 한 곳에서 분석할 수 있도록 Docker 기반 ELK Stack을 구성했습니다. Fleet + Elastic Agent로 중앙 수집 구조를 잡고, Admin/Readonly 권한을 분리했으며, Elastic APM으로 Java 서비스 성능·트랜잭션 모니터링도 구성했습니다.

결과

KISTI 프라이빗 클라우드에 도입했습니다 (2025.02~). VM root 접속을 필수 인원으로 제한하고 그 외 인원은 Kibana를 통해 로그를 조회하도록 전환해 보안을 강화했고, 중앙 집중 조회로 장애·CS 대응 시간을 기존 대비 약 30% 수준으로 단축했습니다. 또한 Elastic APM 슬로우쿼리 로그에서 수신확인 기능의 메일 조회 SQL이 반복 감지되는 것을 발견했습니다. 보낸 메일 전용 기능임에도 전체 메일함 조회 로직을 재사용해 `is\_spam = 0`·`is\_deleted = 0` 조건이 그대로 남아 있어도 없어도 결과가 같은 의미 없는 필터로 작동하고 있었고, 해당 조건을 제거해 쿼리 성능을 개선했습니다.

배운 점

다중 서버를 직접 오가며 로그를 보는 방식의 비효율을 체감하고, 도입 근거와 구성안을 직접 정리해 제안했습니다. 모니터링 가시성이 생기자 기존에는 누구도 인지하지 못했던 슬로우쿼리가 보이기 시작했고, 이를 발견해 직접 개선할 수 있었습니다. 문제는 보여야 해결할 수 있다는 것, 그리고 불편함에서 끝내지 않고 해결 방향을 제시하는 것이 개발자의 역할임을 확인했습니다.

03 고객사 SMS/MMS 외부 API 연동 개발

상황

클라우드 메일 서비스 고객사(한국농수산물유통공사)에서 MMS 기능 추가를 요청했습니다. 기존 SMS 로직이 확장을 고려하지 않은 구조여서 직접 재설계가 필요했습니다.

행동

기존 SMS가 shell curl 기반으로 통신사 API를 호출하고 있어, 통신사 최신 API 문서를 기반으로 RestTemplate 방식으로 재 구현했습니다. 공통 발송 로직과 타입별 처리 로직을 분리해 재설계하고, 통신사 MMS API 연동·이미지 파일 검증·예외 처리를 단독으로 담당했습니다. 엑셀 대량 발송 기능을 추가하며, 대량 발송 시 통신사 1회 요청 수신자 제한을 고려해 체크 단위로 분할하고, CompletableFuture로 병렬 발송 후 전체 결과를 집계해 결과 페이지에 반환하도록 구현했습니다.

결과

한국농수산물유통공사 운영에 적용되었습니다 (2025.06~). 엑셀 기반 대량 발송 도입으로 기존 건별 수동 등록 방식 대비 발송 준비 시간이 크게 단축되었고, SMS 단독 운영에서 MMS까지 발송 가능한 콘텐츠 유형이 확대되었습니다.

배운 점

shell curl로 작성된 레거시 SMS 코드를 RestTemplate으로 재구현하면서, 기존 코드가 왜 그렇게 작성됐는지 파악하는 것부터 시작해야 했습니다. 통신사 API의 수신자 제한이라는 외부 제약을 설계에 반영해 체크 분할·병렬 처리 구조로 대응했고, API 문서가 불충분한 상황에서 직접 문의하며 완성한 경험을 통해 스펙 출처·제약·참고 링크를 코드에 명시하는 문서화 원칙을 세우게 되었습니다.

사용 기술

Spring Framework

Spring Boot 3.x

Java

MySQL

Kafka

Redis

ElasticSearch

ELK Stack

Spring AI

Langfuse

KT Cloud

NCP

Docker

Jenkins

JSP

JavaScript

PORTFOLIO PROJECTS

프로젝트

🔗 포트폴리오

AI Assistant — Backend

Spring Boot 3.5 · Java 21 · Spring AI 1.1.2

Ollama 기반 로컬 LLM 추론 서버와 연동하는 풀스택 AI 어시스턴트 백엔드. RAG · SSE 스트리밍 · 분산 추적 · 이중 캐시를 실제 프로덕션 수준으로 설계 · 구현.

주요 기능

Tool-based RAG

Spring AI Tool Calling으로 RAG를 읊서널하게 실행. 프로젝트 컨텍스트가 필요할 때만 pgvector 유사도 검색 호출

이중 채팅 메모리 (L1/L2)

Redis L1 빠른 조회 + PostgreSQL L2 영구 저장. Spring AI ChatMemoryRepository 인터페이스 직접 구현

SSE 스트리밍 + Idempotency

토큰 단위 스트리밍 전송. X-Idempotency-Key로 중복 요청 차단 — Strategy 패턴으로 4가지 상태 처리

동적 프롬프트 관리

Langfuse에서 런타임에 프롬프트 · 모델 파라미터 조회. 코드 배포 없이 모델 동작 변경 가능

분산 추적 (OTel)

OpenTelemetry 2.17 + Langfuse OTLP로 요청 단위 end-to-end 트레이스. LLM 응답 품질 모니터링

기술 스택

Spring Boot 3.5

Java 21

Spring AI 1.1.2

Ollama

pgvector

Redis

PostgreSQL

OpenTelemetry 2.17

Langfuse

hwplib

핵심 아키텍처 결정 (ADR)

ADR-001

Tool-based RAG vs Naive RAG

매 요청마다 벡터 검색을 수행하면 응답 지연 · 비용 증가. LLM이 컨텍스트 필요 여부를 판단해 Tool을 선택 실행하도록 설계 → 평균 응답 지연 최소화 + OCP 준수

ADR-003

Redis L1 + PostgreSQL L2 이중 메모리

DB 단독 → 응답 지연. Redis 단독 → 재시작 시 맥락 소실. 이중 레이어로 속도와 내구성 동시 확보. Spring AI 인터페이스 직접 구현으로 중복 메시지 방지

ADR-005

Langfuse 동적 프롬프트 관리

프롬프트를 코드에 하드코딩하면 모델 교체 · 튜닝 시 재배포가 필요. Langfuse API에서 런타임 조회 + OpenTelemetry OTLP로 요청 단위 추적까지 통합

트러블슈팅

문서 파싱 (RAG 파이프라인)

Apache Tika + hwplib으로 PDF · HWP · HWPX 파싱. qwen3-embedding:0.6b 임베딩 후 pgvector HNSW 인덱싱

ADR-002

Idempotency — Strategy 패턴

SSE 환경에서 Key 상태(없음/진행중/완료/실패)에 따라 처리 로직이 완전히 달라짐. 4가지 Handler 구현체로 상태별 캡슐화 → SRP + OCP

ADR-004

Java 21 Virtual Thread 전면 활성화

SSE 다중 동시 연결 + Ollama I/O 블로킹.
spring.threads.virtual.enabled=true 설정만으로 OS 스레드 수 상관없이 높은 동시 처리 달성

ChatMemoryRepository 커스텀 — 안정적 ID로 첨부파일 FK 연관관계 구현

문제

Spring AI 기본 JdbcChatMemoryRepository는 메시지 저장 시 대화 전체를 삭제 후 재삽입하여 CHAT_MESSAGE ID가 매 요청마다 재생성. 첨부파일 정보를 저장하는 CHAT_ATTACHMENT 테이블과 FK 연관관계가 불가능한 상황.

해결

ChatMemoryRepository 인터페이스 직접 구현으로 기본 자동 구성 (JdbcChatMemoryRepositoryAutoConfiguration) 비활성화 및 교체. saveAll()을 delete+insert → append-only 방식으로 재정의. type + content 중복 검사 후 신규 메시지만 INSERT하고, 엔티티 생성 시 UUID.randomUUID()로 ID를 1회만 할당하여 불변성 보장.

결과

CHAT_MESSAGE ID 안정화로 FK · JPA @OneToMany 매핑 정상 동작. OCP 원칙에 따라 기존 코드 수정 없이 구현체 교체만으로 해결. 프레임워크 자동 구성 대신 인터페이스 직접 구현이 도메인 요구사항 제어에 더 유리함을 확인.

X-Idempotency-Key 기반 중복 요청 방지 — 네트워크 재시도 시 데이터 일관성 확보

문제

SSE 스트리밍 환경에서 네트워크 불안정으로 인한 클라이언트 재요청 시, 동일한 사용자 메시지가 DB에 중복 저장되고 AI 모델 호출이 중복 발생하여 자원이 낭비되는 문제.

해결

X-Idempotency-Key를 도입하고 Redis를 활용한 상태 관리(IN_PROGRESS, COMPLETED, FAILED) 구현. 재시도 시 (FAILED 상태) 기존 conversationId와 userMessageId를 재사용하여 DB 저장 로직을 스킵하도록 설계.

결과

중복 메시지 발생 0% 달성 및 불필요한 AI 추론 비용 절감. 네트워크 장애 상황에서도 데이터 일관성을 보장하는 신뢰성 있는 스트리밍 서비스 구축.

백엔드 AI Assistant API와 연동하는 Vue 3 프론트엔드를 단독으로 구현했습니다. POST 기반 SSE 스트리밍 수신을 위해 EventSource 대신 Fetch API + ReadableStream을 직접 사용했고, Pinia 없이 Composables 패턴으로 상태를 관리했습니다.

주요 기능

SSE 스트리밍 수신

ReadableStream + 버퍼 누적 방식으로 서버 토큰 스트림 파싱. 클라이언트 재시도 시 동일 Idempotency Key 재사용

AI 사용 대시보드

Langfuse 통계 API 연동. unovis/vue로 일별 사용량 · 비용 · 모델 점유율 차트 시각화

마크다운 렌더링

AI 응답을 markdown-it으로 실시간 파싱 · 렌더링. 코드블록, 표, 목록 지원

Composables 상태 관리

Pinia 없이 useChatState · useAuth · useDashboardData 등 Composable 패턴으로 관심사 분리

Vue Router 없는 SPA

currentView ref로 chat / dashboard / project / pricing 뷰 전환. 불필요한 번들 크기 최소화

다크 / 라이트 테마

useTheme composable + vueuse/core로 OS 설정 감지 및 사용자 토글 지원

기술적 결정

Fetch API 직접 사용 (axios 미사용)

백엔드가 POST + SSE 응답 구조라 GET만 지원하는 EventSource 사용 불가. axios는 브라우저에서 XHR 기반으로 동작해 response.body.getReader()를 지원하지 않아 SSE 청크 실시간 파싱이 불가능. fetch + ReadableStream이 POST 기반 SSE의 브라우저 표준 구현 방식이라 선택.

Pinia 없는 상태 관리

useChatState 호출부가 ChatPage.vue 단 하나라 전역 공유가 불필요. 각 페이지가 독립적으로 동작하며 라우트 간 공유 상태가 없음. useChatState 내부에서 onMounted를 직접 사용하는 패턴은 Pinia 스토어와 근본적으로 맞지 않아 Composable 패턴이 더 적합한 구조.

기술 스택

- Vue 3.5
- TypeScript 5.9
- Vite 7.2
- Tailwind CSS 4
- Shadcn-vue
- Reka UI
- Lucide Vue
- @unovis/vue
- markdown-it
- Composition API
- Fetch API
- SSE (ReadableStream)

SKILLS

기술 스택

Backend

- Java 21
- Spring Boot 3.x
- Spring AI
- Spring Framework
- Redis
- PostgreSQL
- MySQL

Frontend

- Vue 3
- React
- TypeScript
- JavaScript
- Tailwind CSS 4
- Vite
- Shadcn-vue
- JSP

Cloud & DevOps

- KT Cloud
- NCP
- Docker
- Jenkins
- ELK Stack
- Elastic APM
- CI/CD

AI & Observability

- Ollama
- pgvector
- Langfuse
- OpenTelemetry
- RAG